A Voyage to the Kernel

Part 17

Segment: 3.6, Day 16

We have covered the details concerning interrupts and their descriptions during the previous day of our voyage. But one point I deliberately missed was that interrupt handlers form only the first half of the interrupt processing method. The main issue with handlers is that they run asynchronously and may even interrupt with other codes (sometimes even critical ones!). The best way to avoid this is to run the handlers as quickly as possible. You can understand this point more clearly if you can visualise the way in which they deal with your hardware. For this, you may consider handlers as part of a whole mechanism set up to manage actual hardware interrupts. So we can use them for all such time-critical activities.

But we need to route the 'less critical' part to another portion where interrupts are enabled. Thus, managing the interrupts is two-fold. In the first segment (let's call that as 'top half'), the handlers are executed by the kernel asynchronously (as mentioned before) as a response to the hardware interrupt. When it comes to the second part, we deal with actions, which are linked to interrupts that are left out by the handler.

Technically speaking, this second part holds the lion's share of the whole work (since we are giving only 'quick' works to handlers). But the handler does an important job—acknowledging the receipt of interrupts and moving data to/from the hardware. All this is covered in the 'top half'.

Let's take an example. Assume that you want to transfer some data from hardware into memory. This can be handled in the top part and the processing can be done in the bottom part.

This is an important feature as far as the programmer is concerned. This enables the device manager coder to divide the work so as to get the best results. Here are a few guidelines you can use while you write your driver:

- If the work is time-specific (say, you want to finish it soon), use a handler for the work.
- If it is hardware-specific, priority should again be given to the interrupt handler.
- If you wish to handle the processing of any data, then consider the bottom half.

Tip: Before you code your own driver, you are advised to take a look at existing interrupt handlers and bottom halves. And the key point you need to remember is: *the quicker the (handler) execution, the better.*

By using the bottom part, you can limit the work that you intend to do in the handlers since they run with the current interrupt line disabled on all processors. And in the worst case, those